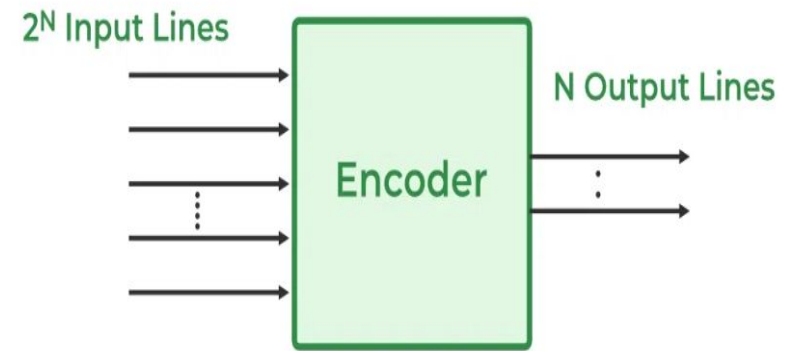


Encoder in Digital Logic

- An encoder is a digital circuit that converts a set of binary inputs into a unique binary code.
- The basic principle of an encoder is to assign a unique binary code to each possible input.
- The output of an encoder is usually active low, meaning that only one output is active (low) at any given time, and the remaining outputs are inactive (high). The active low output is selected based on the binary code assigned to the active input.



Types of Encoders

There are different types of Encoders which are mentioned below.

- 4 to 2 Encoder
- Octal to Binary Encoder (8 to 3 Encoder)
- Decimal to BCD Encoder
- Priority Encoder

4 to 2 Encoder



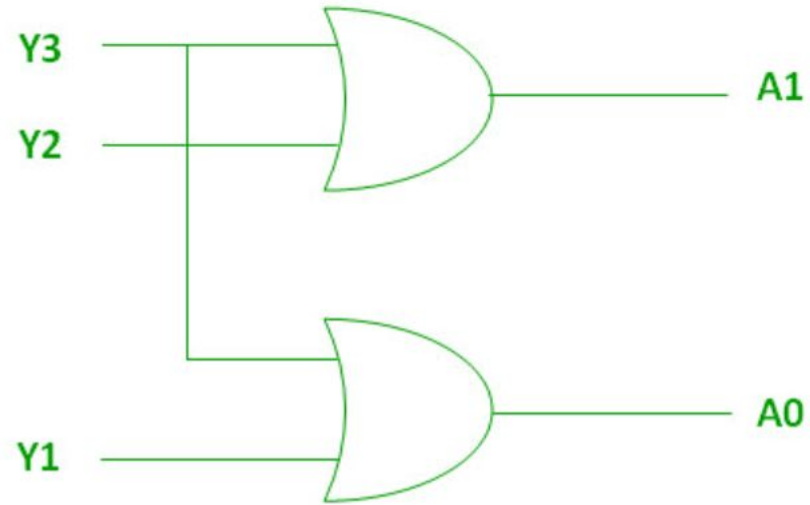
The Truth table of 4 to 2 encoders is as follows.

INPUTS				OUTPUTS	
Y3	Y2	Y1	Y0	A1	A0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Logical expression for A1 and A0:

$$A1 = Y3 + Y2$$

$$A0 = Y3 + Y1$$



```
module encode_4_to_2(  
    input Y0,Y1,Y2,Y3,  
    output A0,A1 );
```

```
    or g2(A0,Y1,Y3);
```

```
    or g1(A1,Y2,Y3);
```

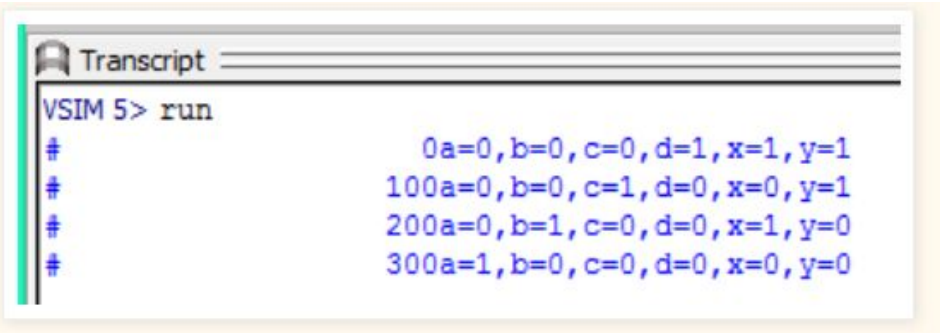
```
endmodule
```

**Write verilog code with gate level
and data flow modeling style**

```

module encoder_4_2(a,b,c,d,x,y);
output x,y;
input a,b,c,d;
assign x = b | d;
assign y = c | d;
endmodule

```



The screenshot shows a VSIM 5 transcript window with the following content:

```

VSIM 5> run
#           0a=0,b=0,c=0,d=1,x=1,y=1
#          100a=0,b=0,c=1,d=0,x=0,y=1
#          200a=0,b=1,c=0,d=0,x=1,y=0
#          300a=1,b=0,c=0,d=0,x=0,y=0

```

```

module encoder_4_2_test;
reg a,b,c,d;
wire x,y;

encoder_4_2 DUT(a,b,c,d,x,y);
initial

begin
#000 a=0; b=0;c=0;d=1;
#100 a=0; b=0;c=1;d=0;
#100 a=0; b=1;c=0;d=0;
#100 a=1; b=0;c=0;d=0;
end
initial
begin
$monitor($time,"a=%b,b=%b,c=%b,d=%b,x=%b,y=%b",a,b,c,d,x,y);
end
endmodule

```

Try out these

Octal to Binary Encoder (8 to 3 Encoder)

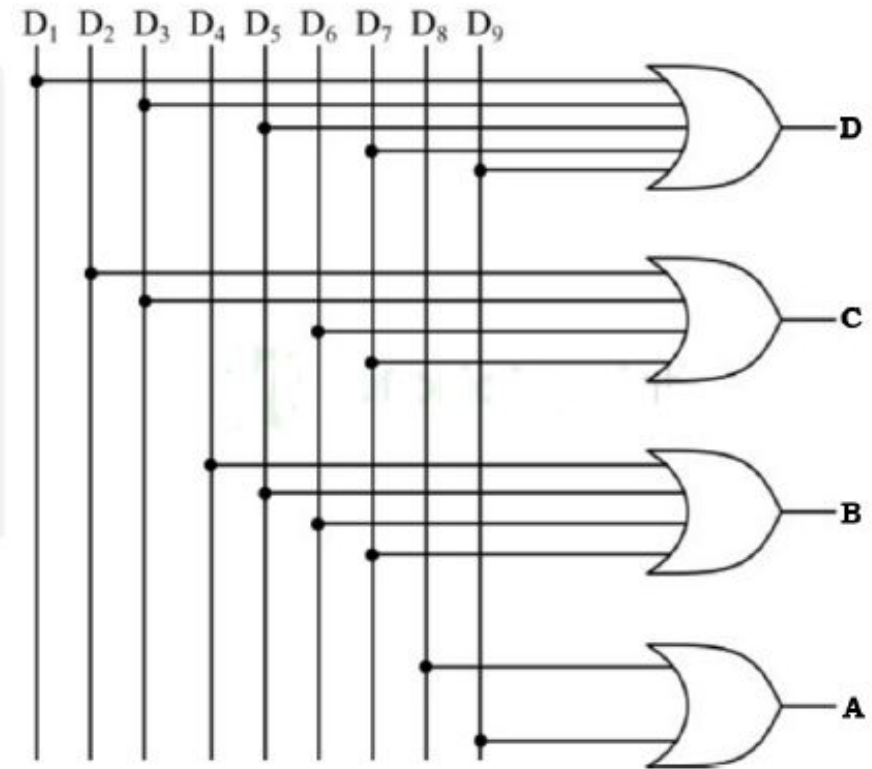
Decimal to BCD Encoder

Decimal to BCD Encoder

Truth Table

Decimal Inputs										BCD Output			
D ₉	D ₈	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀	A	B	C	D
1	0	0	0	0	0	0	0	0	0	1	0	0	1
0	1	0	0	0	0	0	0	0	0	1	0	0	0
0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	0	0	1	0	0	0	0	0	0	0	1	1	0
0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	0	0	1	0	0	0	0

Circuit Diagram



Priority Encoder

INPUTS				OUTPUTS		
Y3	Y2	Y1	Y0	A1	A0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

$$A1 = Y3 + Y2$$

$$A0 = Y3 + Y2' Y1$$

- A priority encoder converts multiple input lines into a binary code on the output lines.
- Unlike a simple encoder, it assigns priority to the inputs — if multiple inputs are active at the same time, the highest-priority input determines the output.

Priority Encoder

INPUTS				OUTPUTS		
Y3	Y2	Y1	Y0	A1	A0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

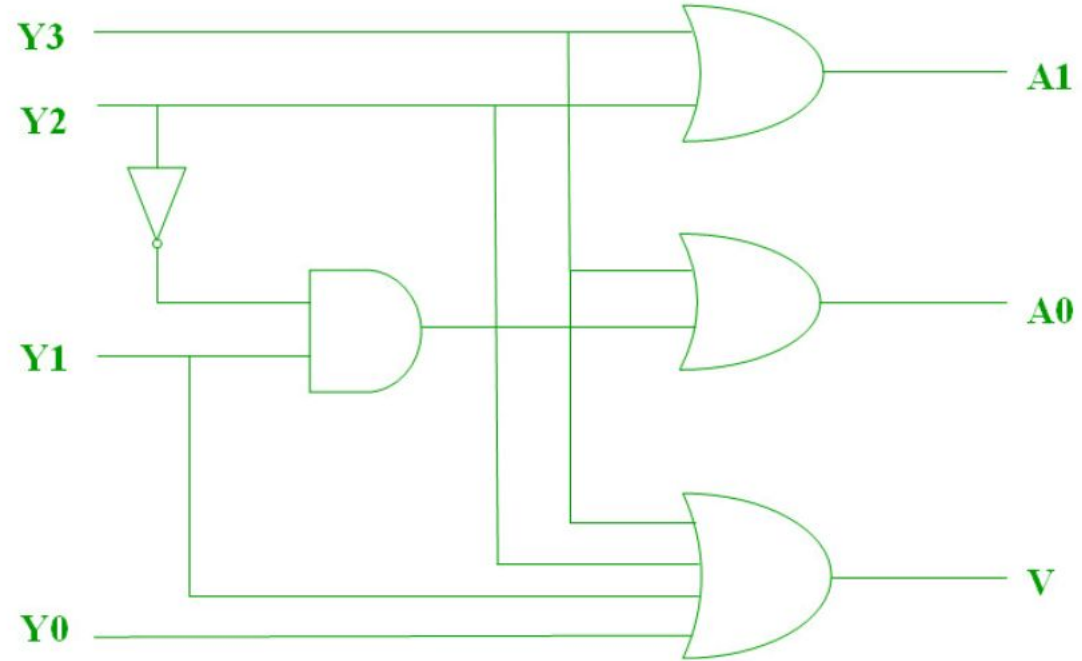
$$A1 = Y3 + Y2$$

$$A0 = Y3 + Y2' Y1$$

Priority Encoder

$$A1 = Y3 + Y2$$

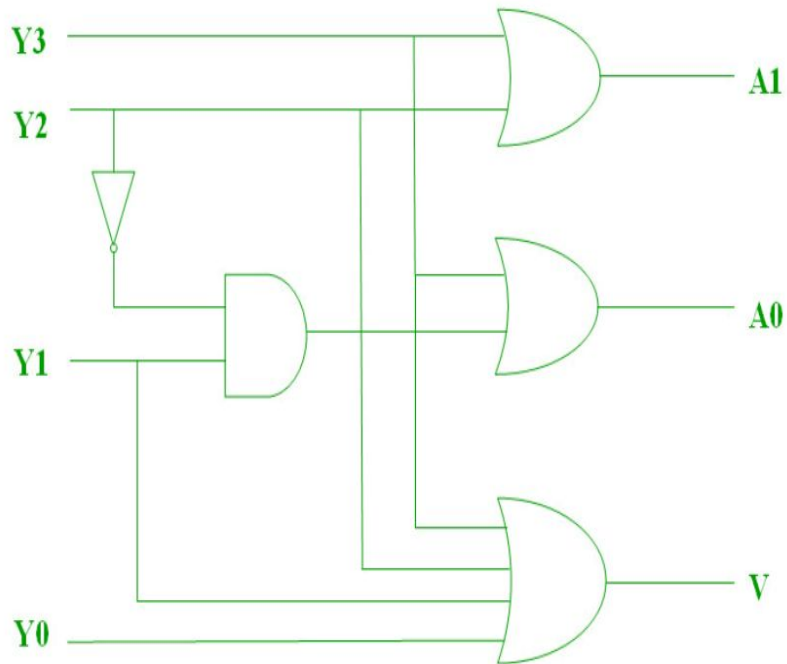
$$A0 = Y3 + Y2' Y1$$



Priority Encoder

$$A1 = Y3 + Y2$$

$$A0 = Y3 + Y2' Y1$$



```
module priority_encoder_42(A0,A1,Y0,Y1,Y2,Y3);
```

```
input Y3, Y2, Y1, Y0;
```

```
output A0, A1;
```

```
wire y2bar; //not of y2
```

```
wire and_out; // and of y2bar and y1
```

```
not(y2bar, y2);
```

```
and(and_out, y2bar, y1);
```

```
or(A1, Y3, Y2);
```

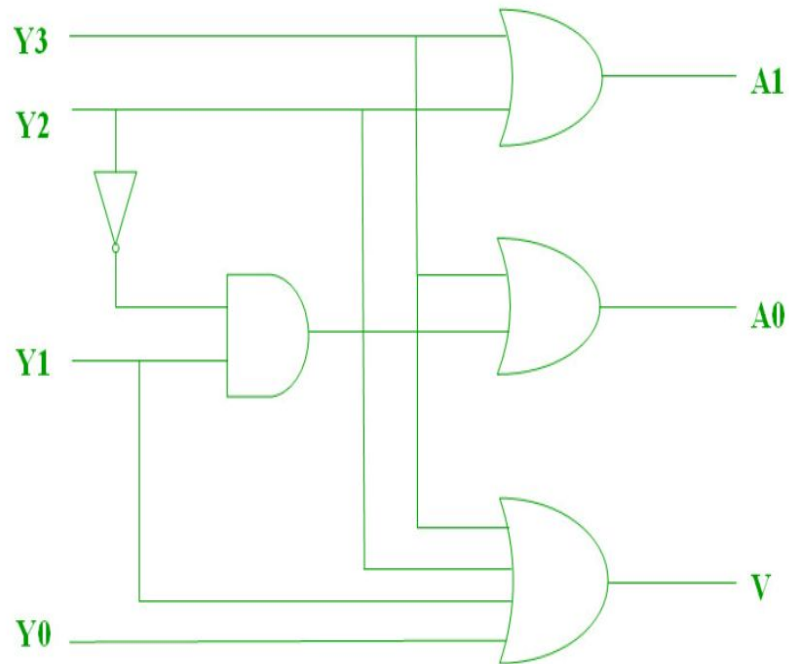
```
or(A0, and_out, Y3);
```

```
endmodule
```

Priority Encoder

$$A1 = Y3 + Y2$$

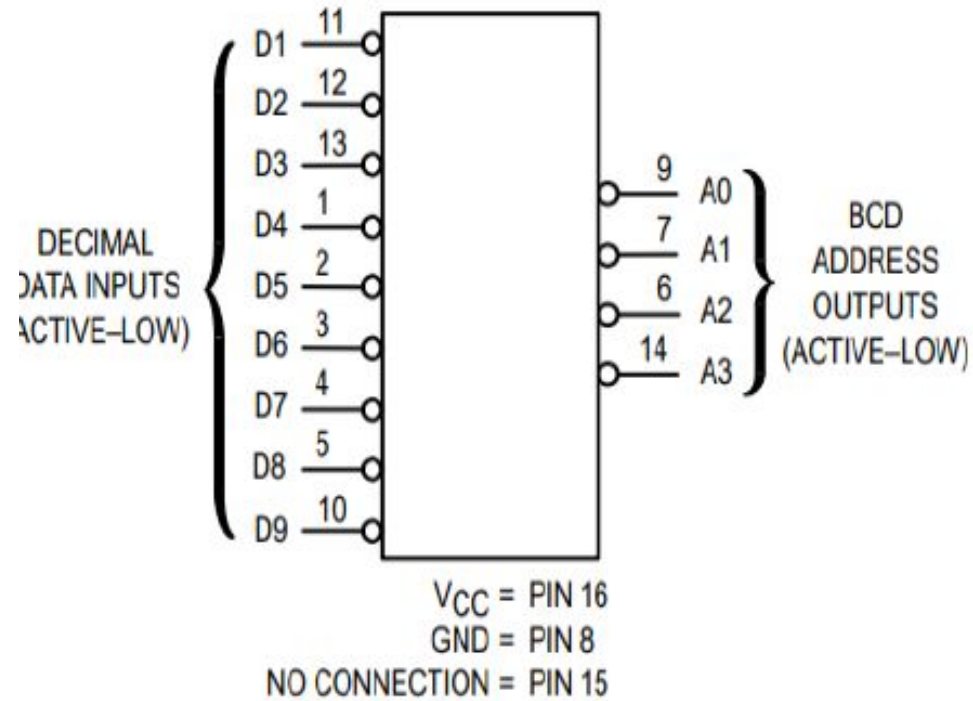
$$A0 = Y3 + Y2' Y1$$



```
module priority_encoder_datafloec(A0,A1,Y0,Y1,Y2,Y3);  
  input Y0,Y1,Y2,Y3;  
  output A0,A1;  
  
  assign A1 = Y3 + Y2;  
  assign A0 = Y3 + ((~Y2)&Y1);  
  
endmodule
```

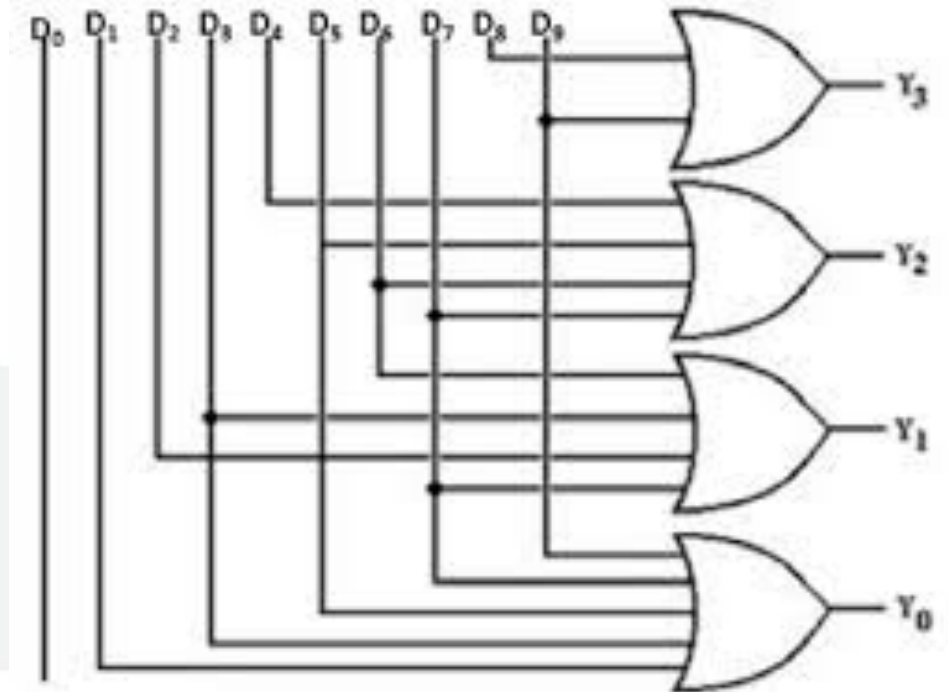
- Example: 74HC147 Decimal to BCD active low priority encoder

Pin Diagram: 74HC147



74HC147:Function Table

Inputs									Output			
D ₉	D ₈	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	A	B	C	D
H	H	H	H	H	H	H	H	H	H	H	H	H
H	H	H	H	H	H	H	H	L	H	H	H	L
H	H	H	H	H	H	H	L	x	H	H	L	H
H	H	H	H	H	H	L	x	x	H	H	L	L
H	H	H	H	H	L	x	x	x	H	L	H	H
H	H	H	H	L	x	x	x	x	H	L	H	L
H	H	H	L	x	x	x	x	x	H	L	L	H
H	H	L	x	x	x	x	x	x	H	L	L	L
H	L	x	x	x	x	x	x	x	L	H	H	H
L	x	x	x	x	x	x	x	x	L	H	H	L



Write Verilog code for Decimal to BCD Priority Encoder using Dataflow modeling

**Write Verilog code for 8:3 Priority Encoder using
Dataflow modeling**

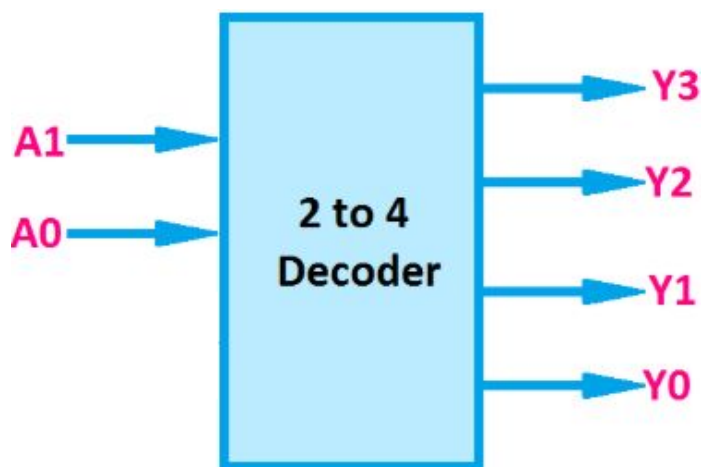
8:3 Priority Encoder

```
module priority_encoder_8_to_3_df ( input wire [7:0] I, output
wire [2:0] y, output wire v );
    assign v = |i;
    //The conditional operator cascades to implement the
    priority.
    assign y = i[7] ? 3'b111 :
                i[6] ? 3'b110 :
                i[5] ? 3'b101 :
                i[4] ? 3'b100 :
                i[3] ? 3'b011 :
                i[2] ? 3'b010 :
                i[1] ? 3'b001 :
                3'b000;
endmodule
```

Decoders

- A decoder converts binary information from n input lines into a maximum of 2^n unique output lines.
- **Example: 2 to 4 Decoder**

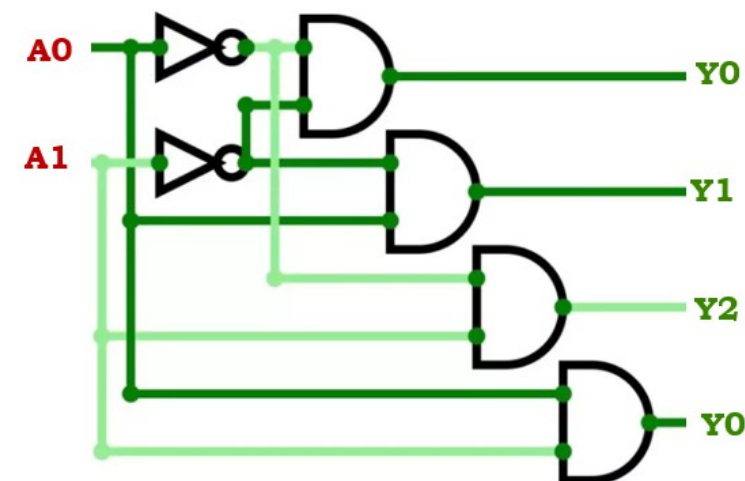
Block Diagram



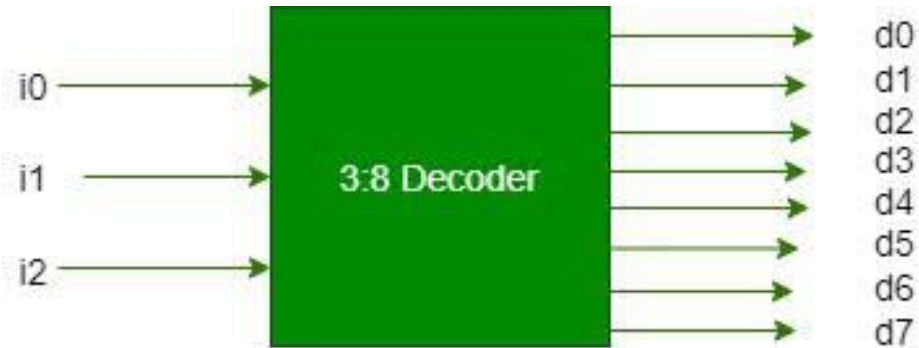
Truth Table

Inputs		Outputs			
A1	A0	Y3	Y2	Y1	Y0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Circuit Diagram



3:8 Decoder



A2	A1	A0	D7	D6	D5	D4	D3	D2	D1	D0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

```
module decoder3x8_dataflow(  
    input [2:0] sel, // 3-bit select input  
    input en,        // Enable input  
    output [7:0] out // 8-bit output  
);  
  
    // Assign statements for dataflow modeling  
    // Each output line is assigned a Boolean expression b  
    assign out[0] = (~sel[2] & ~sel[1] & ~sel[0]) & en;  
    assign out[1] = (~sel[2] & ~sel[1] & sel[0]) & en;  
    assign out[2] = (~sel[2] & sel[1] & ~sel[0]) & en;  
    assign out[3] = (~sel[2] & sel[1] & sel[0]) & en;  
    assign out[4] = ( sel[2] & ~sel[1] & ~sel[0]) & en;  
    assign out[5] = ( sel[2] & ~sel[1] & sel[0]) & en;  
    assign out[6] = ( sel[2] & sel[1] & ~sel[0]) & en;  
    assign out[7] = ( sel[2] & sel[1] & sel[0]) & en;  
  
endmodule
```

Logic functions implementation decoder

A Boolean function with n variables can be implemented using a decoder with n inputs.

Example: Implementation of a full subtractor using 3 to 8 decoder

Truth table

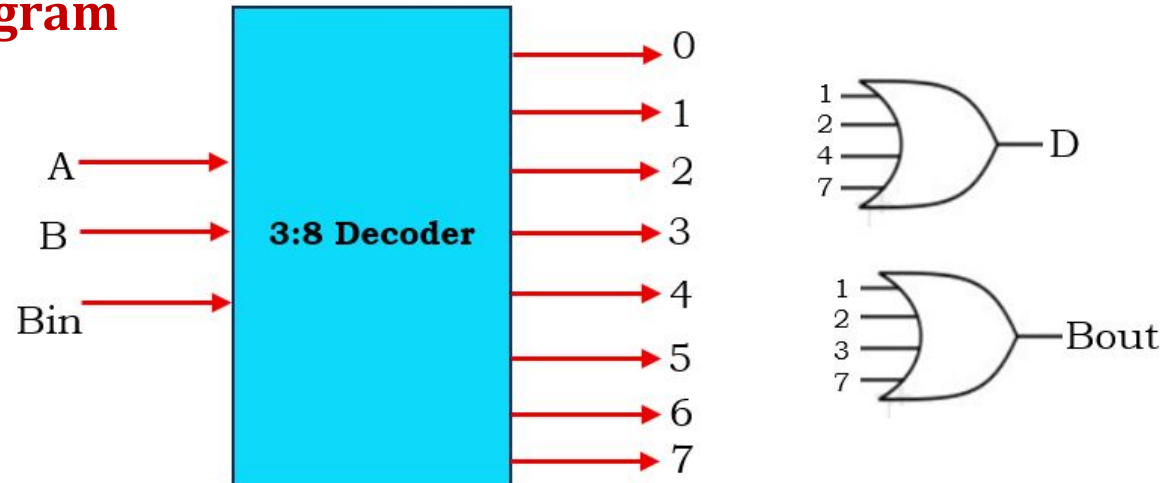
A	B	Bin	Diff (D)	Borrow (Bout)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Minterms

$$D(A,B,Bin) = \sum m(1,2,4,7)$$

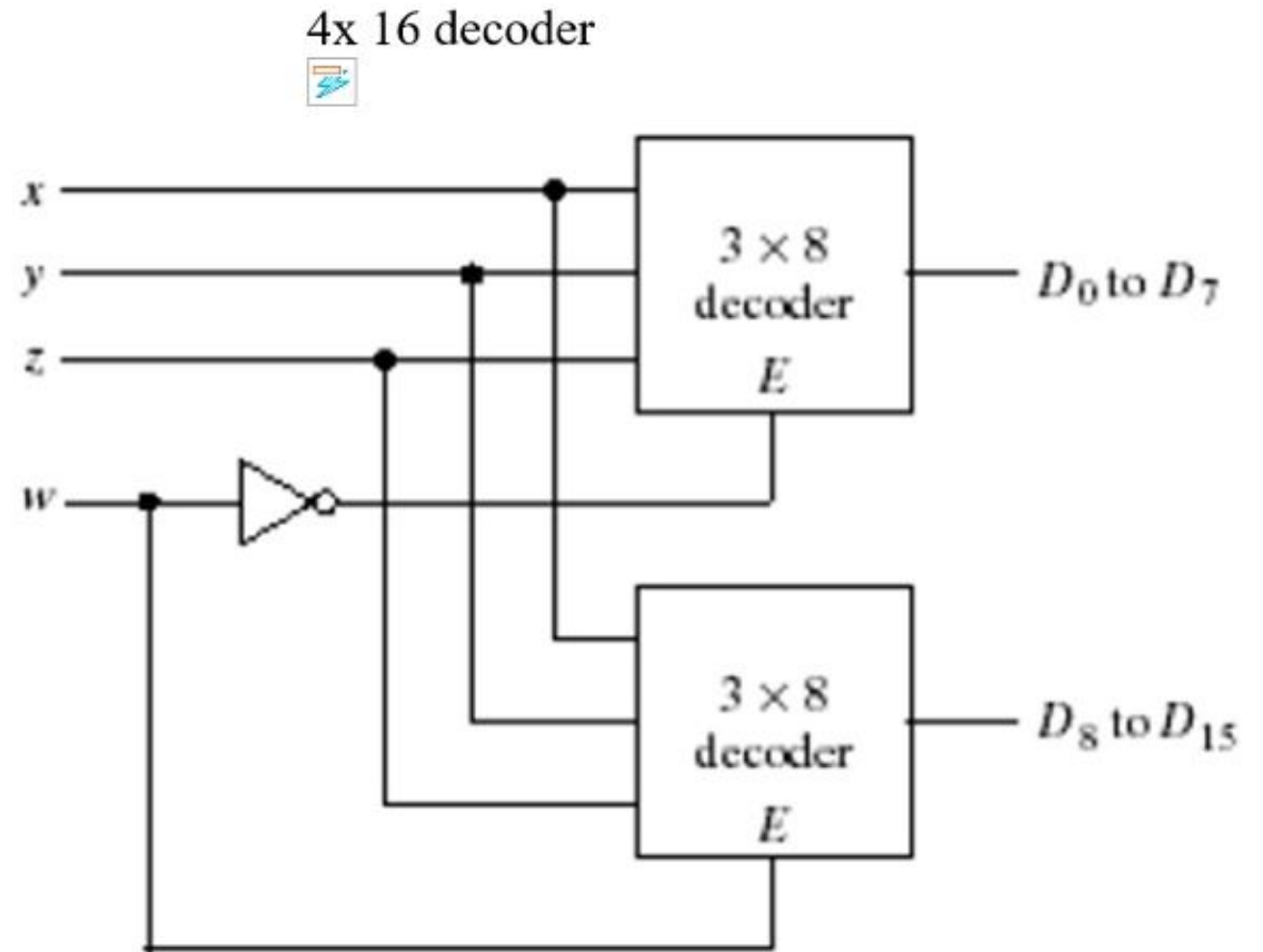
$$Bout(A,B,Bin) = \sum m(1,2,3,7)$$

Logic Diagram

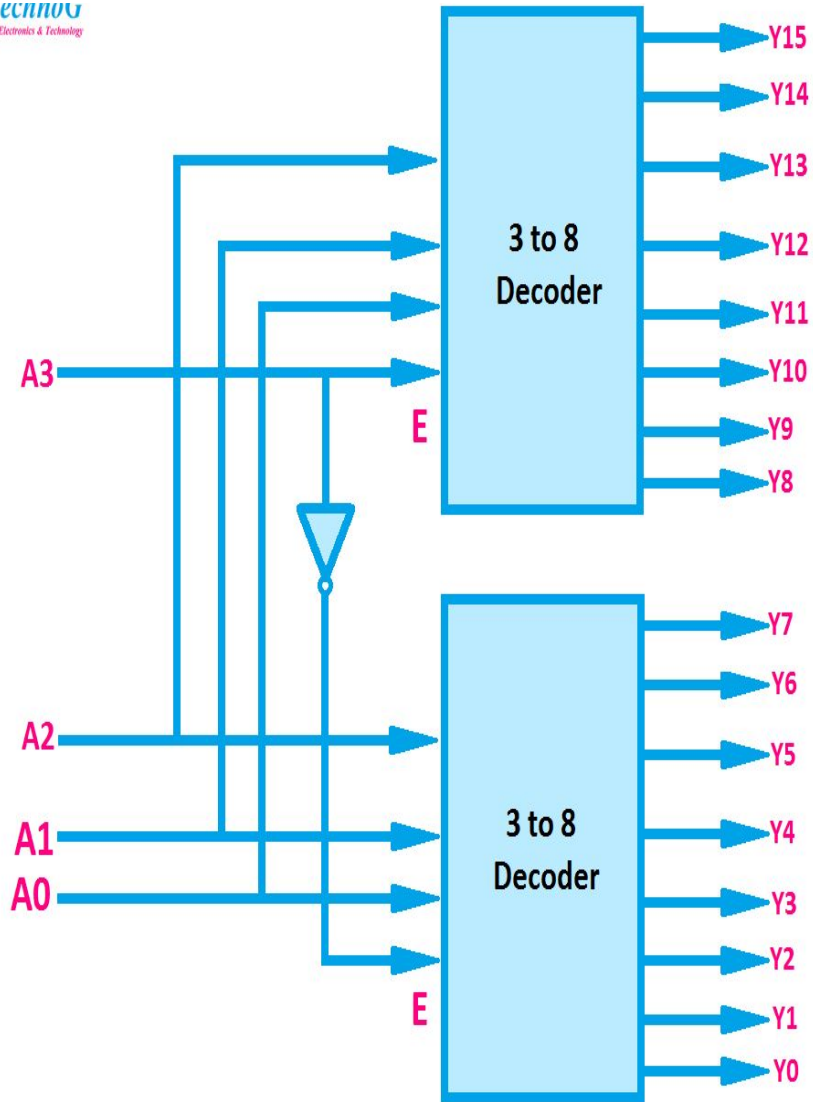


4: 16 decoder using 3:8 decoder

4:16 Decoder					
En	Inputs				Output
1	0	0	0	0	D0
1	0	0	0	1	D1
1	0	0	1	0	D2
1	0	0	1	1	D3
1	0	1	0	0	D4
1	0	1	0	1	D5
1	0	1	1	0	D6
1	0	1	1	1	D7
1	1	0	0	0	D8
1	1	0	0	1	D9
1	1	0	1	0	D10
1	1	0	1	1	D11
1	1	1	0	0	D12
1	1	1	0	1	D13
1	1	1	1	0	D14
1	1	1	1	1	D15



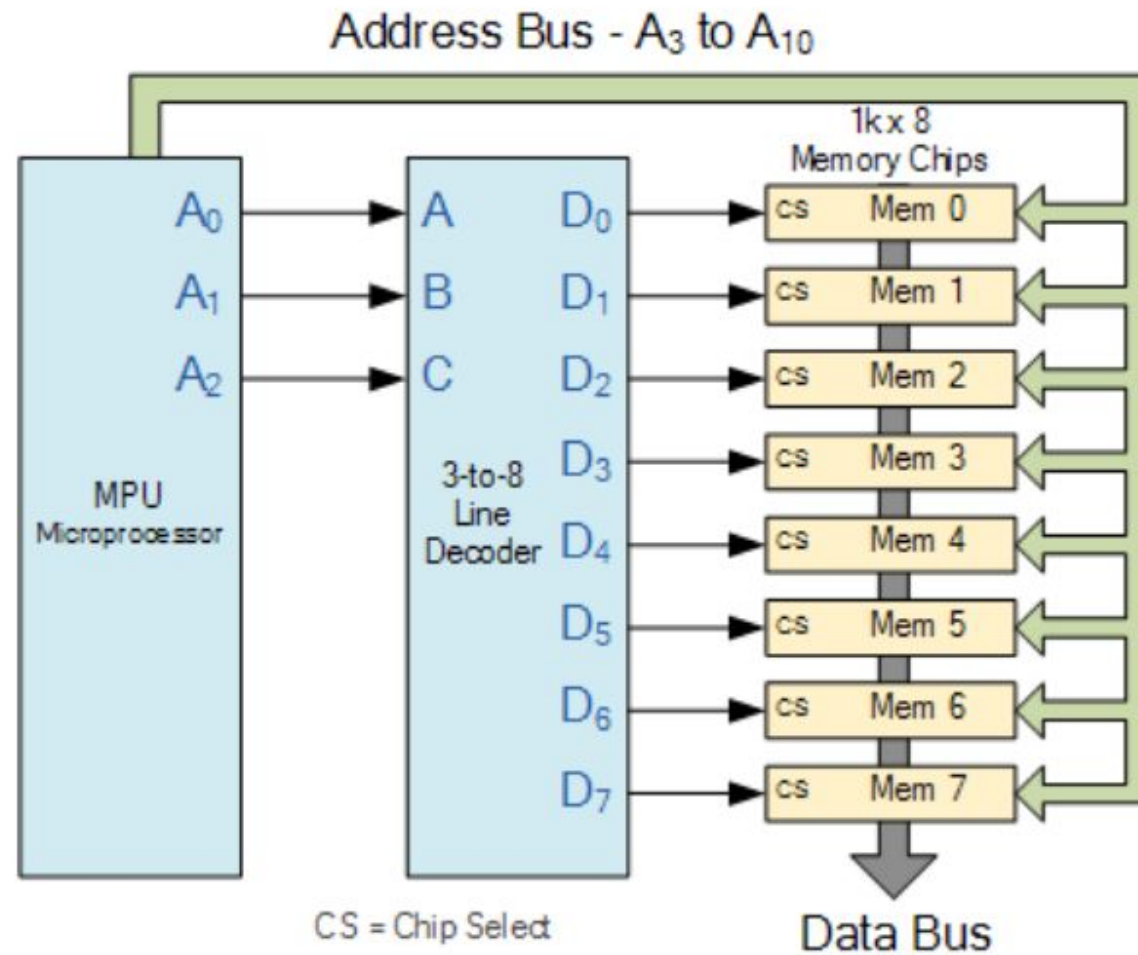
4: 16 decoder using 3:8 decoder- Verilog Code



```
module decoder4_to_16 (  
    input [3:0] A,  
    output [15:0] D  
);  
  
    // The Lower 3 bits of A are connected to the A input of both decoders  
    // The MSB (A[3]) is used as the enable signal for the first decoder  
    // The inverse of the MSB is used as the enable signal for the second decoder  
    decoder3_to_8 dec1 (  
        .A(A[2:0]),  
        .E(A[3]),  
        .D(D[7:0]) // Connect to the lower half of the output  
    );  
  
    decoder3_to_8 dec2 (  
        .A(A[2:0]),  
        .E(~A[3]),  
        .D(D[15:8]) // Connect to the upper half of the output  
    );  
endmodule
```

Application of decoder

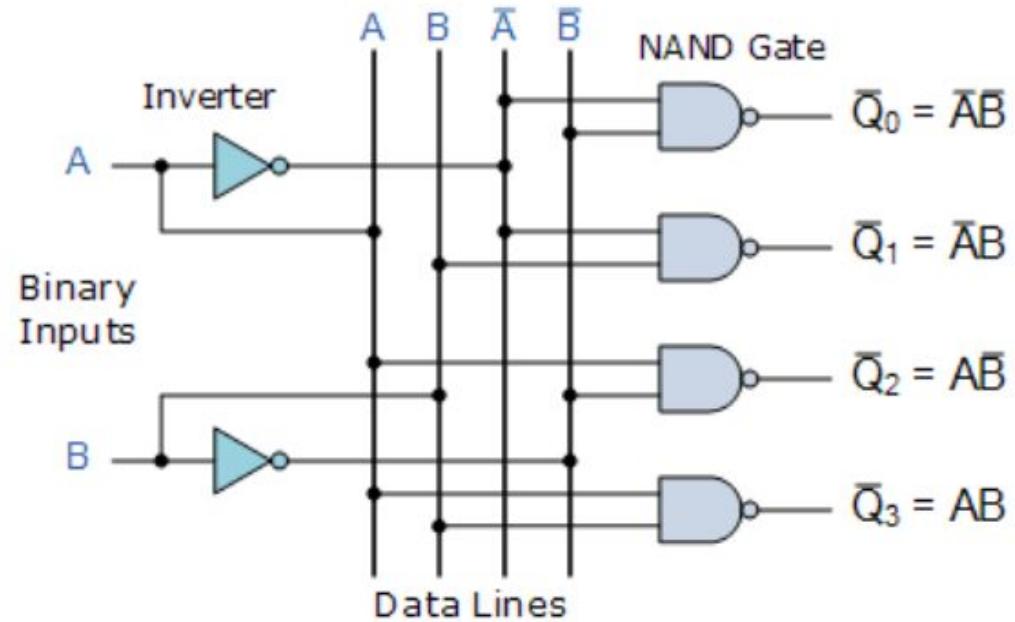
Memory Address Decoding



2-to-4 Line NAND Binary Decoder– ACTIVE LOW

Truth Table

A	B	Q_0	Q_1	Q_2	Q_3
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0



Question 9

Realize a 5 to 32 decoder using 3 to 8 decoders. Show the function table.

Logic functions implementation using Multiplexers

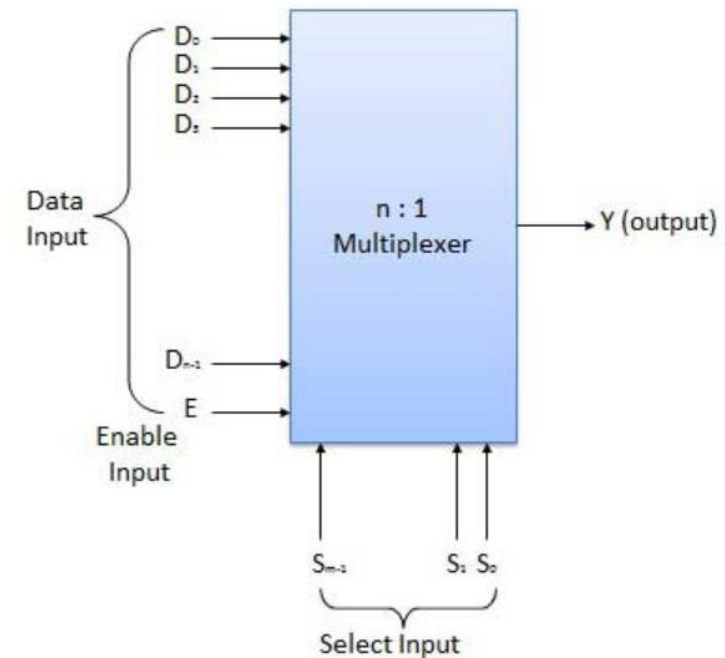
- A multiplexer (MUX) is a combinational circuit that selects one of many input signals and forwards the selected input to a single output line.

- **Basic terms:**

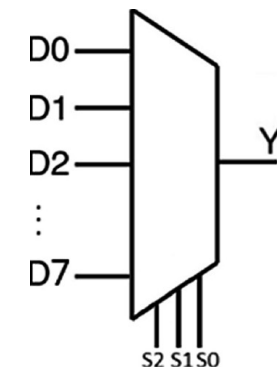
n:1 MUX — n is data inputs, 1 output. Select lines, and choose which input is connected to the output.

Enable (optional) — an active-high/low input that allows the mux to drive the output only when enabled

**Block
Diagram**



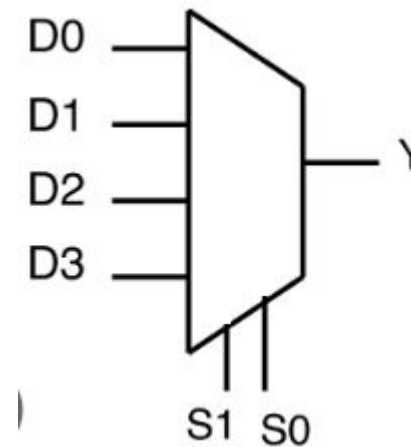
**Symbol
(8:1)**



Logic functions implementation using Multiplexers

- A Multiplexer can be used to implement any Boolean function by selecting the appropriate combination of inputs.
- If a Boolean expression has (n+1) variables, then 'n' of these variables can be connected to the select lines of the multiplexer. The remaining single variable, along with constants 1 and 0, is used as the input of the multiplexer.
- In general, a Boolean expression of (n+1) variables can be implemented using a

4:1 Multiplexer



Truth table

S1 S0	Y
0 0	D0
0 1	D1
1 0	D2
1 1	D3

Logic Equation

$$Y = \overline{S1} \overline{S0} D0 + \overline{S1} S0 D1 + S1 \overline{S0} D2 + S1 S0 D3$$

Logic functions implementation using 4:1 Mux

- **Example: $F(A, B, C) = \sum m(1, 3, 5, 6)$**
- **Implementation table:**

Apply variables B and C to the select lines. The procedures for implementing the function are:

- List the input of the multiplexer
- List under them all the minterms in two rows, with the first half of the minterms with $\bar{A}$ and the second half with A.

	D_0	D_1	D_2	D_3
$\bar{A}$	0	1	2	3
A	4	5	6	7

Logic functions implementation using 4:1 Mux

The given function is implemented by circling the minterms of the function and applying the following rules to find the values for the inputs of the multiplexer.

- If both the minterms in the column are not circled, apply 0 to the corresponding input.
- If both the minterms in the column are circled, apply 1 to the corresponding input.
- If the only bottom minterm is circled, apply A to the input.

	D ₀	D ₁	D ₂	D ₃
$\bar{A}$	0	①	2	③
A	4	⑤	⑥	7
	0	1	A	$\bar{A}$

From the table,

$$D_0=0, D_1=1, D_2=A, D_3=A$$

Implementation of $F(A, B, C) = \sum m(1, 3, 5, 6)$ using 4:1 Mux

Logic Diagram



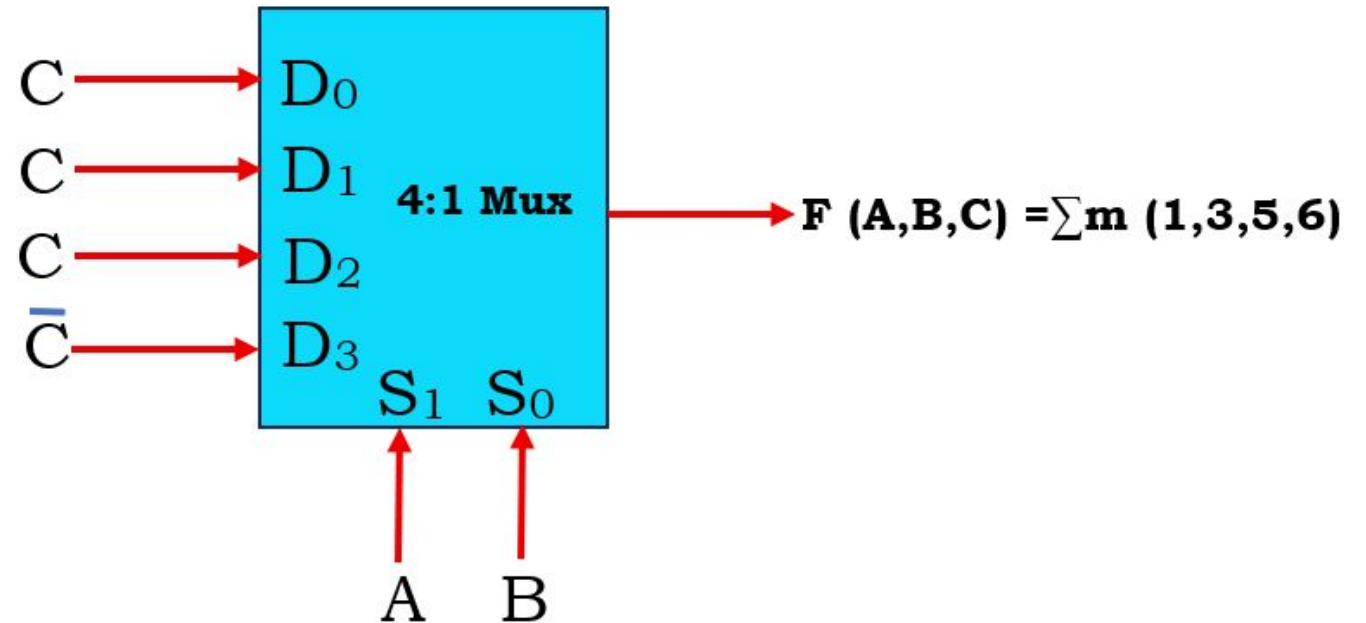
Implementation of $F(A, B, C) = \sum m(1, 3, 5, 6)$
using 4:1 Mux

- Apply variables A and B to the select lines.

Implementation Table

	D_0	D_1	D_2	D_3
$\bar{C}$	0	2	4	⑥
C	①	③	⑤	7
	C	C	C	$\bar{C}$

Logic Diagram



Question 5: Implement the following Boolean function using MUX

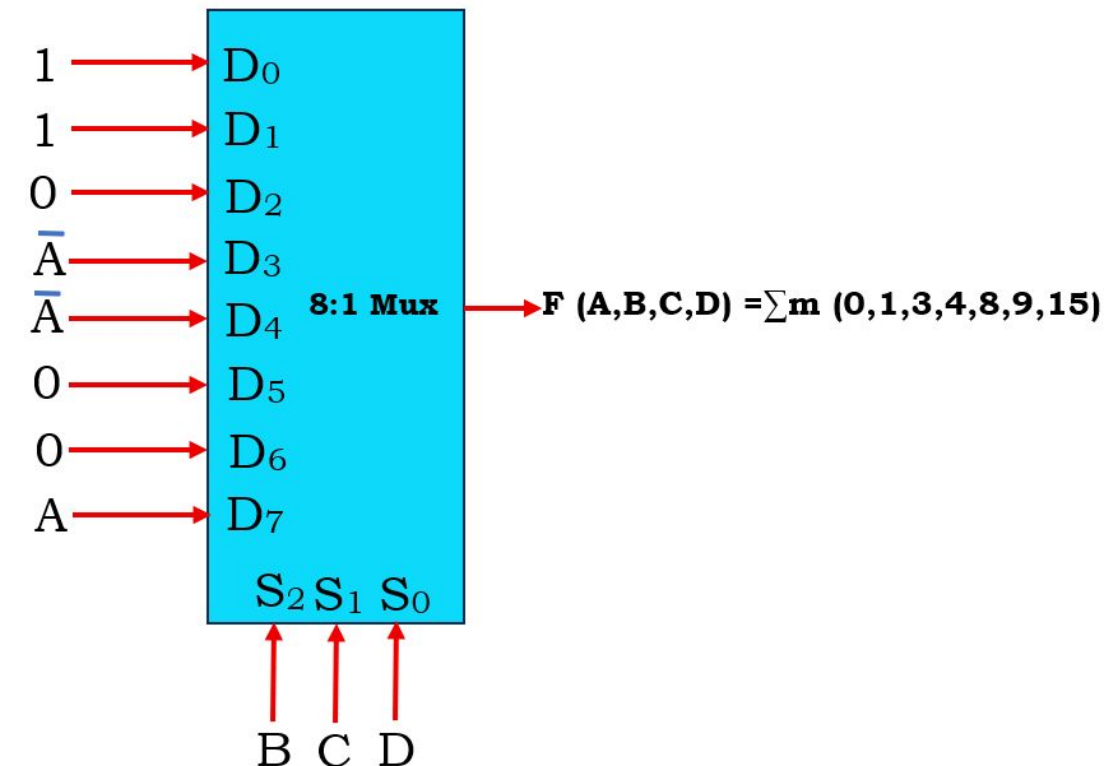
$$F(A,B,C,D) = \sum m(0,1,3,4,8,9,15)$$

- Assuming A is connected to inputs and B,C,D are connected to select lines.

Implementation Table

	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
$\bar{A}$	0	1	2	3	4	5	6	7
A	8	9	10	11	12	13	14	15
	1	1	0	$\bar{A}$	$\bar{A}$	0	0	A

Logic Diagram



Question: Implement the following Boolean function using MUX (4 :1)

$$F(A,B,C,D) = \sum m(1,2,5,7,9,14,15)$$

Question: Implement the following Boolean function using MUX (8 :1)

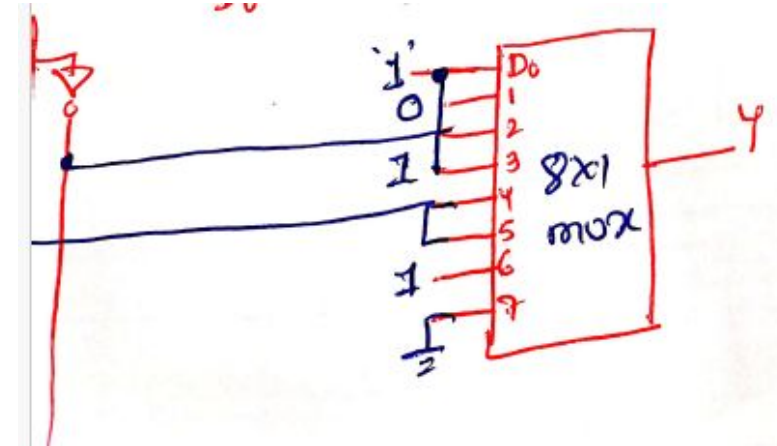
$$F(A,B,C,D) = \sum m(0,2,6,11,12,13) + d(3,8,14)$$

Question: Implement the following Boolean function using MUX (8 :1)

$$F(A,B,C,D) = \sum m(0,2,6,11,12,13) + d(3,8,14)$$

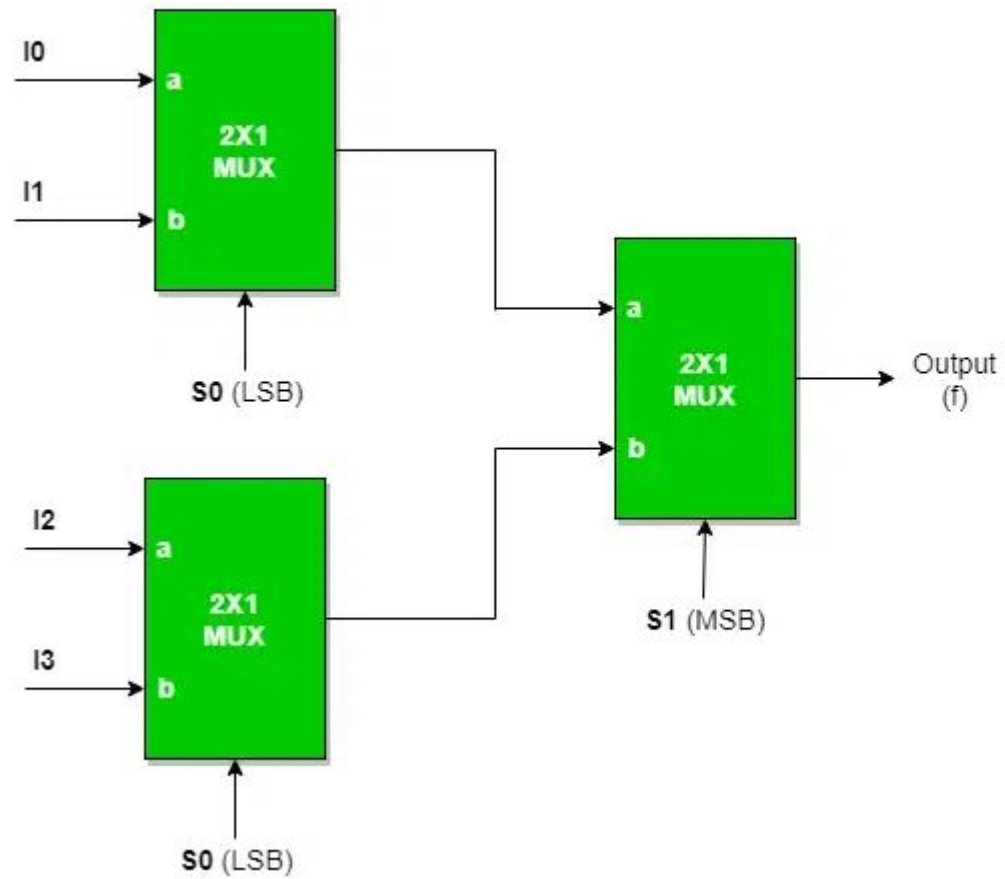
Implementation Table:

	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
$\bar{A}$	0	1	2	3	4	5	6	7
A	8	9	10	11	12	13	14	15
	1	0	$\bar{A}$	1	A	A	1	0
	↓	↓	↓	↓	↓	↓	↓	↓
	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7



Higher Order MUX using LOWER Order MUX

Inputs



Truth Table

S1	S0	f
0	0	I0
0	1	I1
1	0	I2
1	1	I3

Verilog Code

```
// Module for a 2:1 multiplexer
module mux_2to1 (
    input  I0,
    input  I1,
    input  S,
    output Y
);
    assign Y = S ? I1 : I0;
endmodule
```

```
// Module for a 4:1 multiplexer using three 2:1 muxes
module mux_4to1_using_2to1 (
    input  I0,
    input  I1,
    input  I2,
    input  I3,
    input  S1,
    input  S0,
    output Y
);
```

```
    wire mux1_out, mux2_out;
```

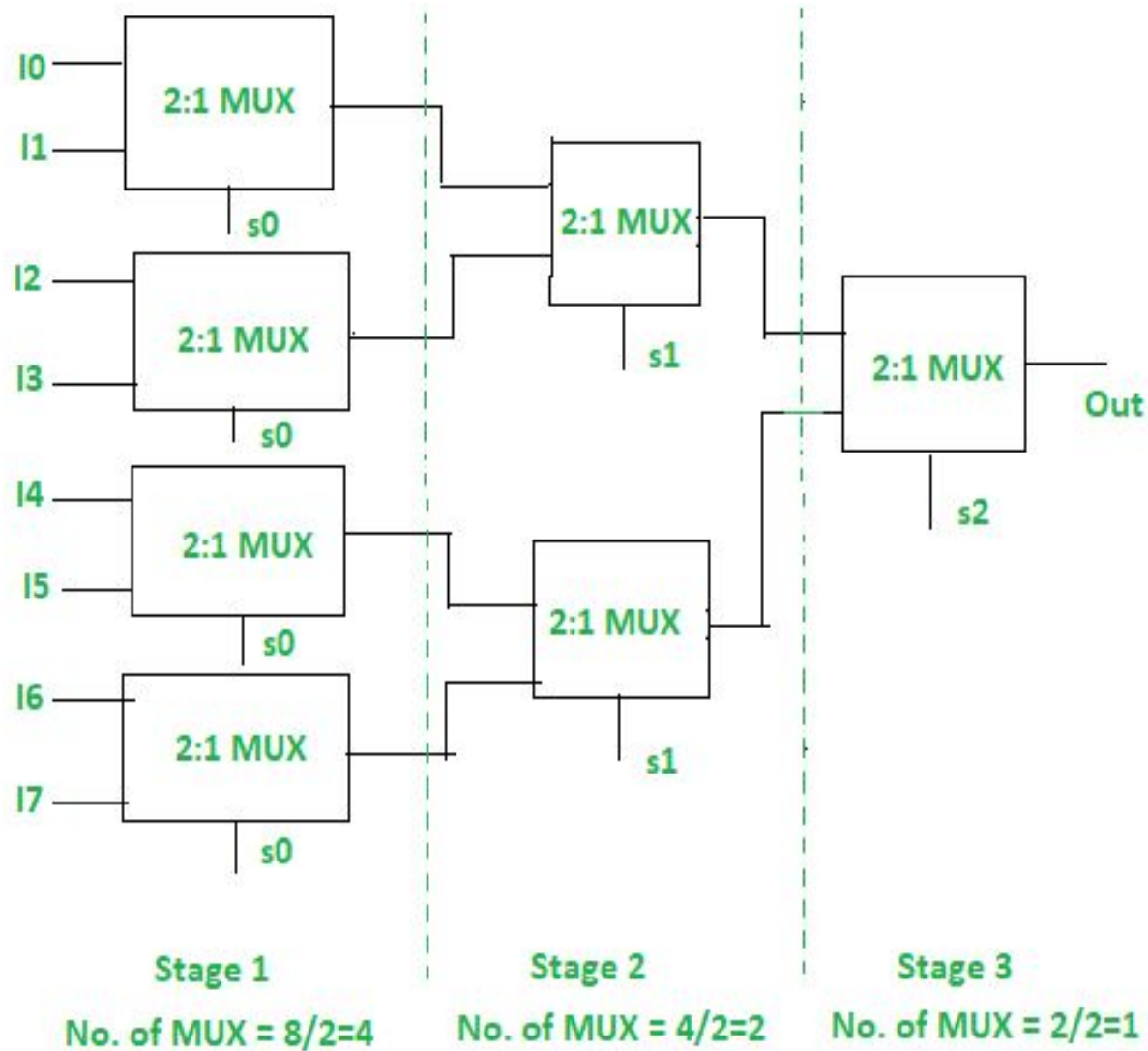
```
// Instantiate the first 2:1 mux
mux_2to1 mux1 (
    .I0(I0),
    .I1(I1),
    .S(S0),
    .Y(mux1_out)
);
```

```
// Instantiate the second 2:1 mux
mux_2to1 mux2 (
    .I0(I2),
    .I1(I3),
    .S(S0),
    .Y(mux2_out)
);
```

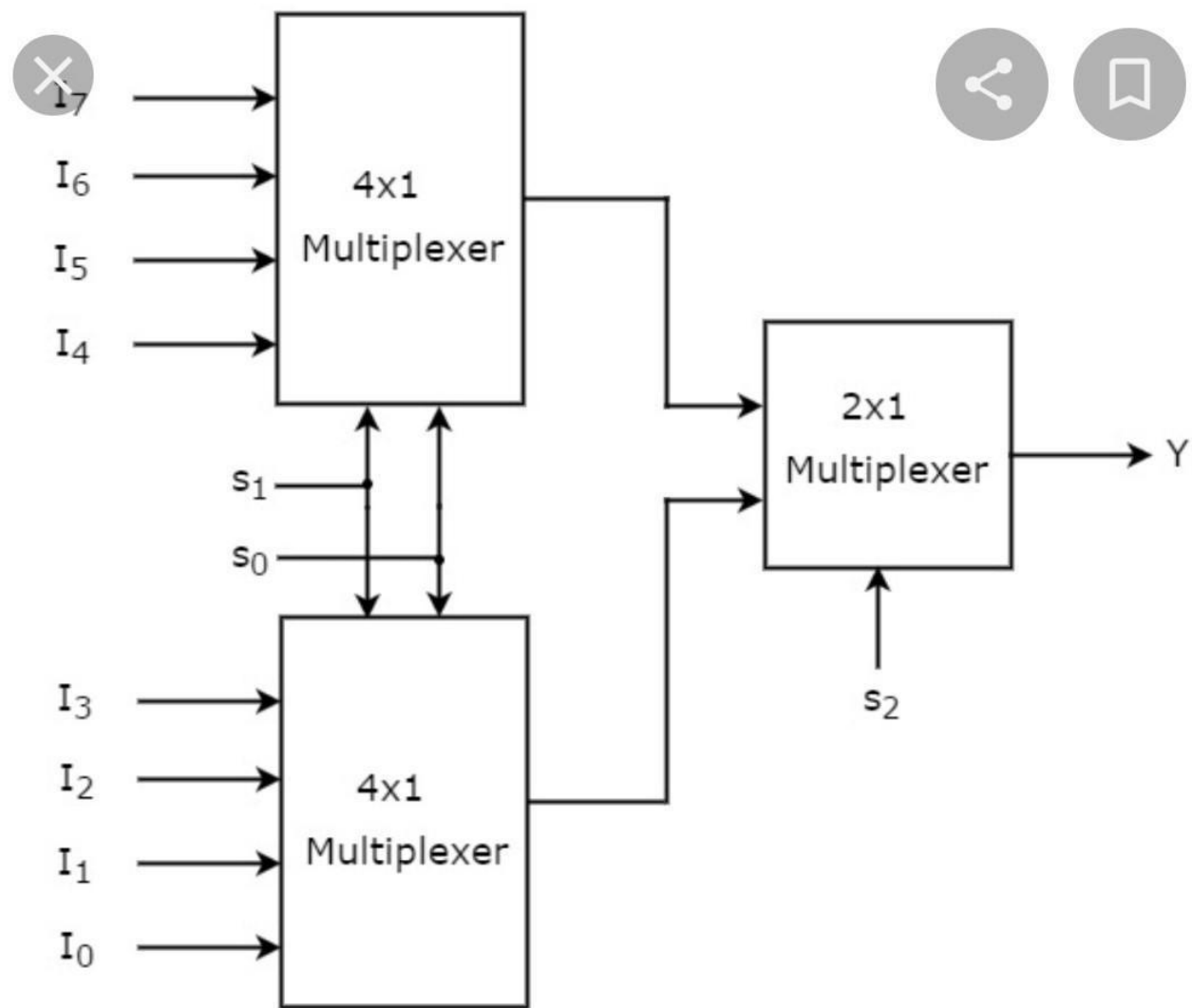
```
// Instantiate the third 2:1 mux to combine the results
mux_2to1 mux3 (
    .I0(mux1_out),
    .I1(mux2_out),
    .S(S1),
    .Y(Y)
);
```

```
endmodule
```

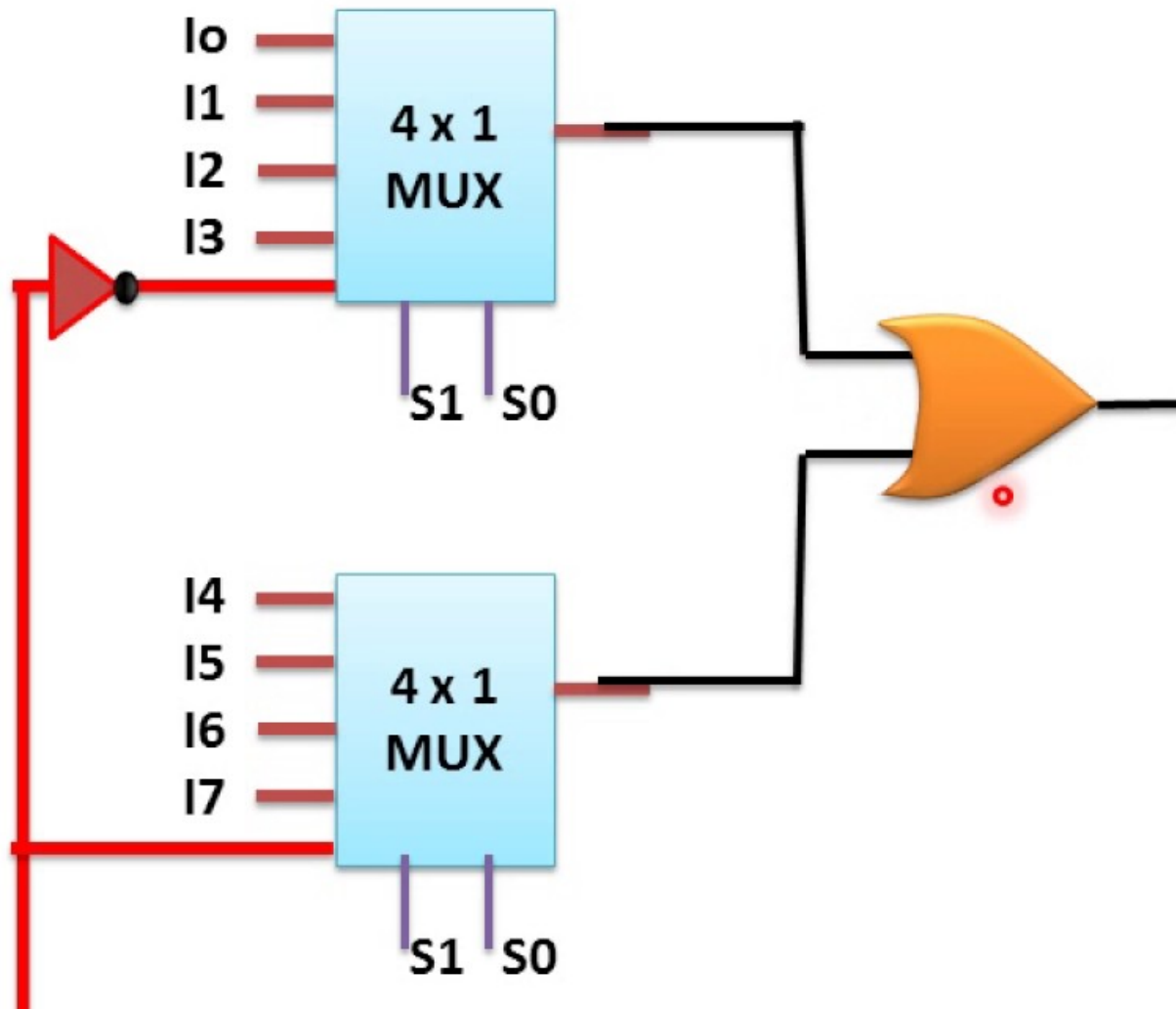
8:1 MUX using 2:1 MUX



INPUTS			Output
S ₂	S ₁	S ₀	Y
0	0	0	A ₀
0	0	1	A ₁
0	1	0	A ₂
0	1	1	A ₃
1	0	0	A ₄
1	0	1	A ₅
1	1	0	A ₆
1	1	1	A ₇



Design 8:1 mux using only TWO 4:1 Mux (Can use additional gates) Write Verilog Code.



S2	S1	S0	Y
0	0	0	I0
0	0	1	I1
0	1	0	I2
0	1	1	I3
1	0	0	I4
1	0	1	I5
1	1	0	I6
1	1	1	I7

$$\begin{aligned}
 n &= 8 \\
 8/4 &= 2 \\
 2/4 &= 0.5 \quad + \quad = 2.5
 \end{aligned}$$

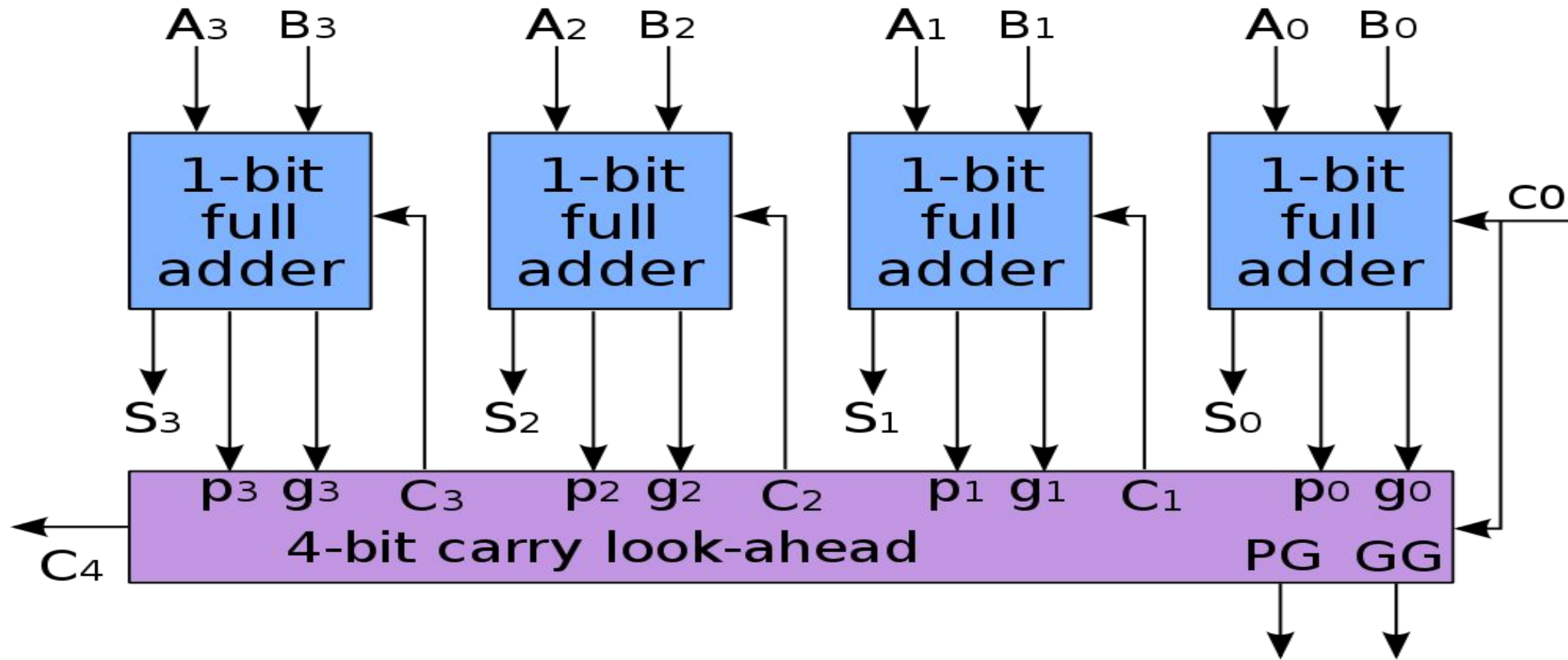
Delay in Ripple Carry Adder

- In ripple carry adders, for each adder block, the two bits that are to be added are available instantly. However, each adder block waits for the carry to arrive from its previous block.
- So, it is not possible to generate the sum and carry of any block until the input carry is known.
- The i^{th} block waits for the $i-1^{\text{th}}$ block to produce its carry. So there will be a considerable time delay which is carry propagation delay.

Carry Look Ahead Adder

- A Carry Look-Ahead (CLA) adder is a type of adder circuit that aims to improve the speed of addition by minimizing the propagation delay of carry signals.
- It's a parallel adder that can add multiple bits simultaneously by generating carry signals in advance, rather than waiting for them to propagate through the adder stages.
- A carry look-ahead adder reduces the propagation delay by introducing more complex hardware
- The carry signals are "looked ahead" to reduce the overall computation time.

A block diagram of a 4-bit carry lookahead adder



Here a carry signal will be generated in two cases:

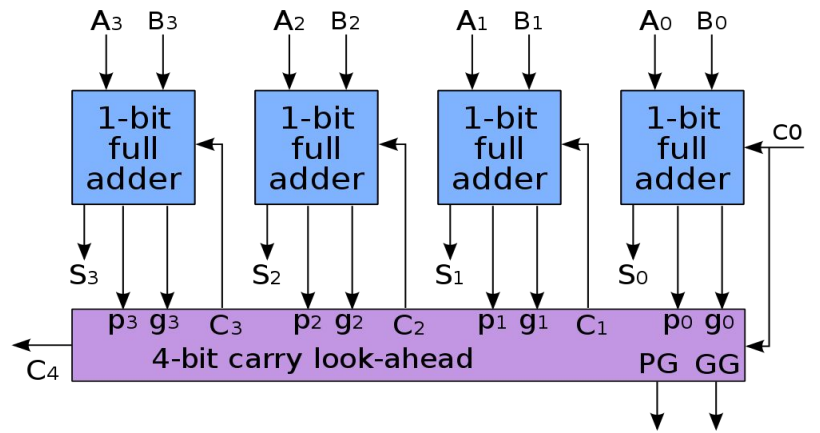
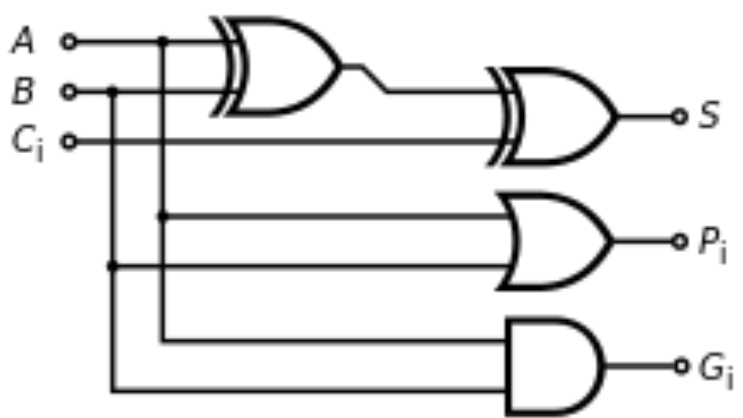
- 1. Input bits A and B are 1
- 2. When one of the two bits is 1 and the carry-in is 1.

Generates carry if and only if both A and B are 1

$G(A, B) = A \cdot B$

Propagates if and only if at least one of A or B is 1

$P(A, B) = A + B$



A	B	C _i	C _o	Type of Carry
0	0	0	0	None
0	0	1	0	None
0	1	0	0	None
0	1	1	1	Propagate
1	0	0	0	None
1	0	1	1	Propagate
1	1	0	1	Generate
1	1	1	1	Generate/Propagate

A	B	C_i	C_o	Type of Carry
0	0	0	0	None
0	0	1	0	None
0	1	0	0	None
0	1	1	1	Propagate
1	0	0	0	None
1	0	1	1	Propagate
1	1	0	1	Generate
1	1	1	1	Generate/Propagate

$$C_0 = \underbrace{A \cdot B}_{\text{carry gen.}} + \underbrace{(A \oplus B) \cdot C_i}_{\text{carry prop.}}$$

$$C_{i+1} = G_i + (P_i \cdot C_i).$$

$$C_1 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + P_1 \cdot C_1$$

$$C_3 = G_2 + P_2 \cdot C_2$$

$$C_4 = G_3 + P_3 \cdot C_3$$

$$C_1 = G_0 + P_0 \cdot C_0$$

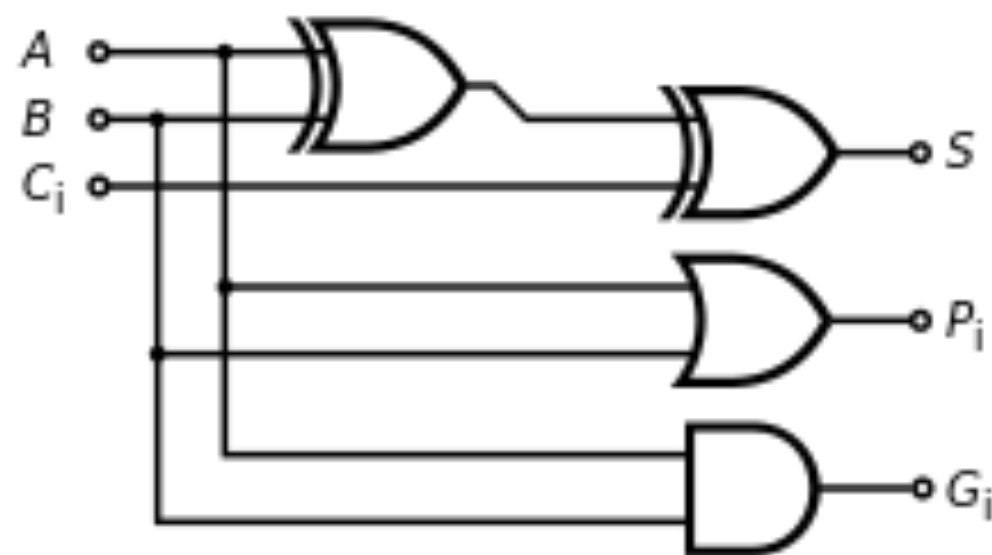
$$C_2 = G_1 + G_0 \cdot P_1 + C_0 \cdot P_0 \cdot P_1$$

$$C_3 = G_2 + G_1 \cdot P_2 + G_0 \cdot P_1 \cdot P_2 + C_0 \cdot P_0 \cdot P_1 \cdot P_2$$

$$C_4 = G_3 + G_2 \cdot P_3 + G_1 \cdot P_2 \cdot P_3 + G_0 \cdot P_1 \cdot P_2 \cdot P_3 + C_0 \cdot P_0 \cdot P_1 \cdot P_2 \cdot P_3$$

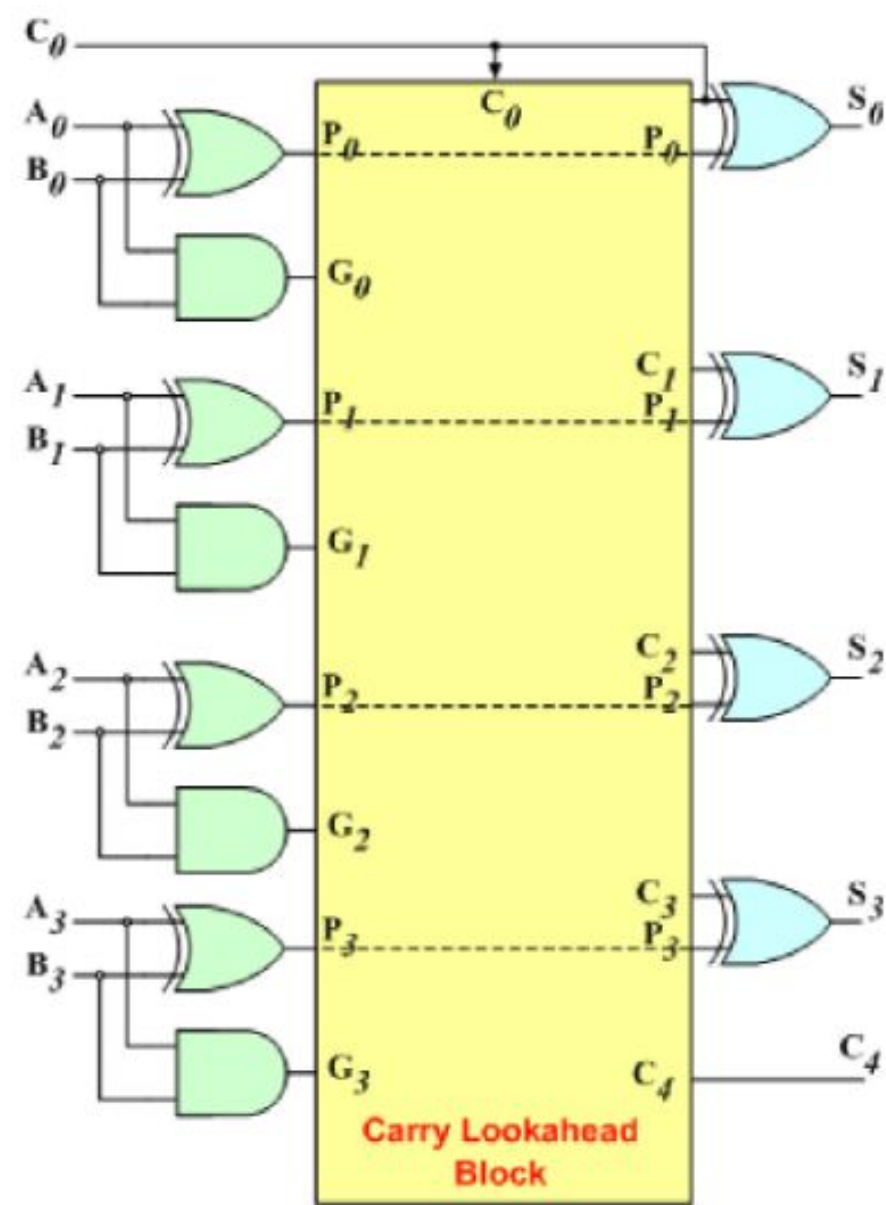
$$C_1 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + G_0 \cdot P_1 + C_0 \cdot P_0 \cdot P_1$$



$$C_3 = G_2 + G_1 \cdot P_2 + G_0 \cdot P_1 \cdot P_2 + C_0 \cdot P_0 \cdot P_1 \cdot P_2$$

$$C_4 = G_3 + G_2 \cdot P_3 + G_1 \cdot P_2 \cdot P_3 + G_0 \cdot P_1 \cdot P_2 \cdot P_3 + C_0 \cdot P_0 \cdot P_1 \cdot P_2 \cdot P_3$$



Carry Look Ahead Adder

Code:

```
module carry_look_adder (input [3:0] a, b, input cin, output [3:0] sum, output cout );

wire g0,p0,g1,p1,g2,p2,g3,p3;
wire c1,c2,c3;
assign g0 = a[0] & b[0];
assign p0 = a[0] ^ b[0];
assign sum[0] = a[0] ^ b[0] ^ cin;

assign c1 = g0 | (p0 & cin);
assign g1 = a[1] & b[1];
assign p1 = a[1] ^ b[1];
assign sum[1] = a[1] ^ b[1] ^ c1;

assign c2 = g1 | (p1 & c1);
assign g2 = a[2] & b[2];
assign p2 = a[2] ^ b[2];
assign sum[2] = a[2] ^ b[2] ^ c2;

assign c3 = g2 | (p2 & c2);
assign g3 = a[3] & b[3];
assign p3 = a[3] ^ b[3];
assign sum[3] = a[3] ^ b[3] ^ c3;

assign cout = g3 | (p3 & c3);
endmodule
```